GLOBALRAIN

Practices for Secure Software Report

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	6/21/26	Elijah Bastien	

Client

Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Elijah Bastien

1. Algorithm Cipher

Based on Artemis Financial's recent security assessment, the Advanced Encryption Standard (AES) algorithm using the Galois/Counter Mode (GCM) of operation with NoPadding is recommended to modernize the application's defensive posture. Artemis's internal dependency-check report highlights critical cryptographic and timing vulnerabilities coming from outdated third-party libraries. Its reliance on an obsolete Bouncy Castle library (bcprov-jdk15on version 1.46) and an old Apache Tomcat container (version 9.0.30) exposes the environment to high profile Side Channel and Timing Flaws tracked under CVE-2023-33202, CVE-2024-30171, and CVE-2026-43514. These flaws create observable timing discrepancies and data leakage, which malicious actors can exploit to reconstruct sensitive cryptographic operations.

AES is a symmetric key block cipher adopted by the U.S. government and recognized globally. In symmetric encryption, the same key is used for both encryption and decryption, offering rapid processing times ideal for financial transactions. This contrasts with asymmetric encryption (public/private key pairs), which is mathematically slower and typically reserved for digital signatures and initial TLS handshakes. A bit-level of 256 bits (AES-256) provides a total keyspace of 2^256 possible combinations, rendering brute force attacks mathematically impossible with current computing standards.

GCM is selected as the mode of operation because it is an Authenticated Encryption with Associated Data (AEAD) mode. Unlike traditional block modes like Cipher Block Chaining (CBC), GCM handles both confidentiality and integrity checks simultaneously by appending an authentication tag. This directly mitigates Artemis's systemic timing issues and protects the application from padding oracle attacks. Lastly, NoPadding is specified because GCM functions like a stream cipher under the hood by generating a keystream, meaning it does not require data blocks to be padded to fixed multiples, eliminating padding mechanism vulnerabilities entirely.

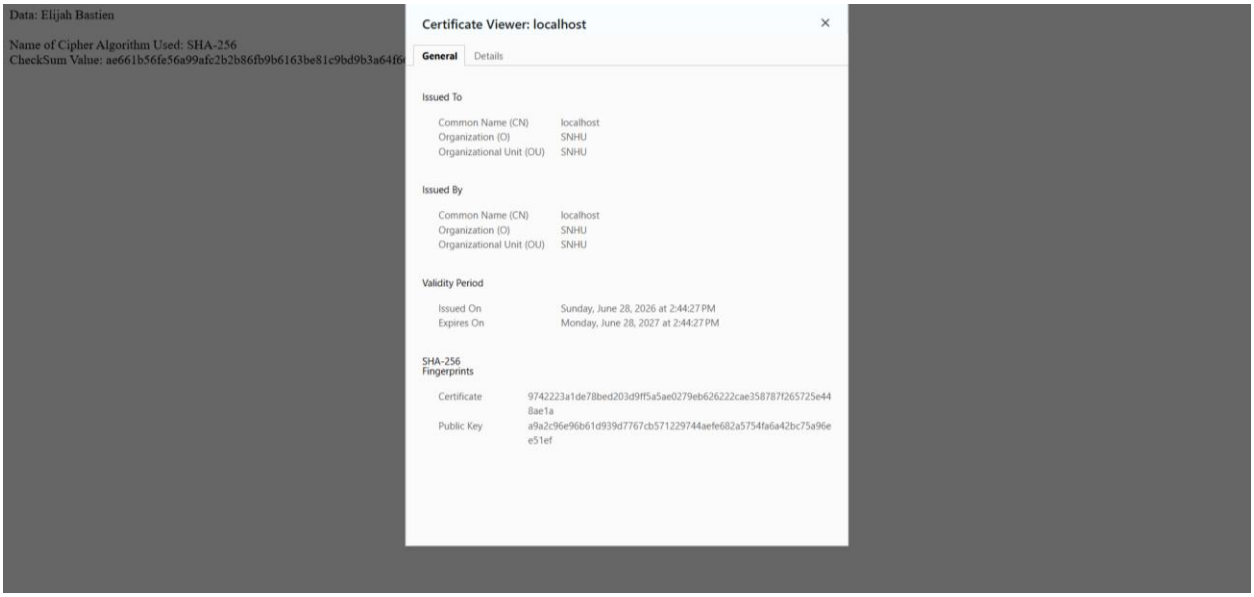
2. Certificate Generation

```
PS C:\Users\Elijah> & "C:\Program Files\Java\jdk-26.0.1\bin\keytool.exe" -genkeypair -alias selfsigned -keyalg RSA -keysize 2048 -storetype JKS -keystore ke
ystore.jks -storepass Haitibyentomkuesa -keypass Haitibyentomkuesa -validity 365
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
  [Unknown]:  localhost
What is the name of your organizational unit?
  [Unknown]:  SNHU
What is the name of your organization?
  [Unknown]:  SNHU
What is the name of your City or Locality?
  [Unknown]:  Boston
What is the name of your State or Province?
  [Unknown]:  MA
What is the two-letter country code for this unit?
  [Unknown]:  MA
Is CN=localhost, OU=SNHU, O=SNHU, L=Boston, ST=MA, C=MA correct?
  [no]:  yes

Generating 2,048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=localhost, OU=SNHU, O=SNHU, L=Boston, ST=MA, C=MA
```

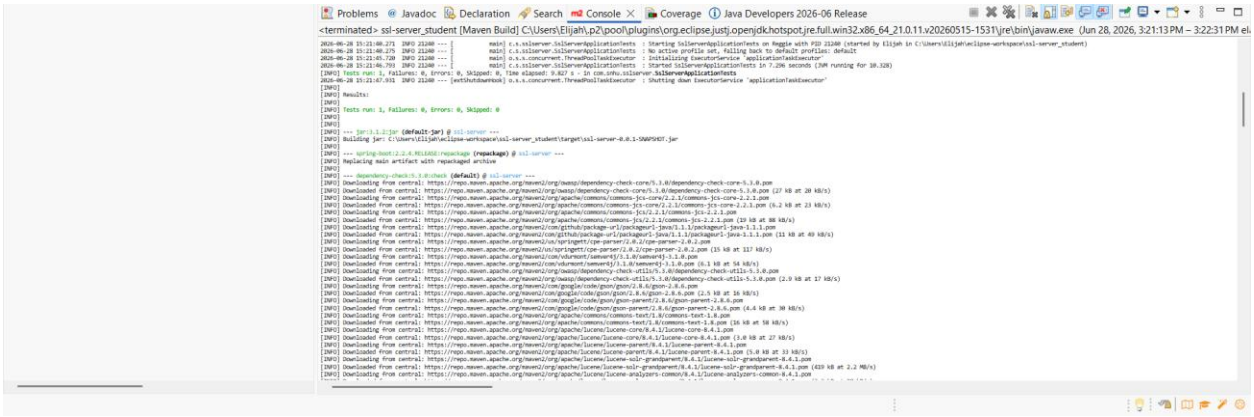
3. Deploy Cipher

4. Secure Communications



5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.



Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Coun
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncy-castle-crypto-package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_crypto_package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_for_java:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncy-castle-java-crytography-api:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:the_bouncy_castle_crypto_package_for_java:1.46:*:*:*:*:*:*:*	pkg:maven/org.bouncycastle/bcprov-jdk15on@1.46	HIGH	3
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	3
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	6
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	3
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	8
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	17
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	18
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	60
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	61

* indicates the dependency has a known exploited vulnerability

Dependencies (vulnerable)

bcprov-jdk15on-1.46.jar

Description:

The Bouncy Castle Crypto package is a Java implementation of cryptographic algorithms. This jar contains JCE provider and lightweight API for the Bouncy Castle Cryptography APIs for JDK 1.5 to JDK 1.7.

License:

Bouncy Castle Licence: http://www.bouncycastle.org/licence.html

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.

```java
package com.snhu.sslserver;

import java.security.MessageDigest;

@SpringBootApplication
public class SslServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SslServerApplication.class, args);
    }
}

@RestController
class ServerController {

    @RequestMapping("/hash")
    public String myHash() {
        // Static data
        String data = "Elijah Bastien";

        try {
            // Create SHA-256 MessageDigest object
            MessageDigest digest = MessageDigest.getInstance("SHA-256");

            // Generate hash bytes
            byte[] hashBytes = digest.digest(data.getBytes());

            // Convert hash bytes to a hexadecimal string
            String checksum = bytesToHex(hashBytes);

            // Return formatted response showing data, algorithm, and checksum
            return "Data: " + data + "<br><br>"
                    + "Name of Cipher Algorithm Used: SHA-256<br>"
                    + "CheckSum Value: " + checksum;

        } catch (NoSuchAlgorithmException e) {
            return "Error generating checksum.";
        }
    }

    // Helper method to convert byte array to Hexadecimal string
    private String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
```

7. Summary

The refactoring of Artemis Financial's web interface addresses fundamental security flaws by embedding proactive, modern defensive mechanisms. To mitigate the risk of data interception and unauthorized manipulation, communication protocols were transitioned from plaintext HTTP to encrypted HTTPS on port 8443 using a localized keystore. Additionally, data integrity verification was established by deploying a SHA-256 cryptographic hash function. This ensures that any data transferred through the interface receives a unique digital fingerprint, allowing both clients and internal endpoints to instantaneously detect data tampering or packet corruption.

As a developer, my responsibilities throughout this process included diagnosing architectural flaws via software dependency analysis, generating trusted localized credentials via Java Keytool, and refactoring Java REST controllers to support secure formatting. Software security directly impacts the business value of an application. For a financial consulting firm like Artemis, security breaches result in devastating regulatory penalties, loss of consumer trust, and intellectual property theft. Implementing cryptographic validation and transport-layer encryption preserves the company’s market reputation, fulfills strict financial compliance mandates (such as FIPS 140-3), and ensures the long term reliability of its digital ecosystem.

8. Industry Standard Best Practices

To maintain a strong defensive posture, Artemis Financial must incorporate secure coding lifecycle practices and continuously monitor dependencies. The discovery of high-severity vulnerabilities in the application's Bouncy Castle and Apache Tomcat elements demonstrates that open-source dependencies change over time. Artemis should implement automated scanning tools (such as OWASP Dependency-Check or Snyk) directly into their continuous integration and continuous deployment (CI/CD) pipelines to catch outdated libraries before code reaches production. The initial code analysis revealed systemic code vulnerabilities like plaintext database passwords and unprotected variables. Industry standard best practices dictate utilizing runtime environment variables or external configuration managers (such as AWS Secrets Manager or Spring Cloud Vault) to separate sensitive administrative keys from the source code. Lastly while self signed certificates are acceptable for testing environments, production infrastructures must use certificates issued by a globally recognized Certificate Authority (CA). CAs prevent Man in the Middle attacks through domain validation and ensure native browser recognition. This guarantees an encrypted channel while eliminating the alarming security warning pop ups that degrade client confidence.